CSE 2421

Lecture 2.2

CSE 2421

I/O

Basic I/O

- There is no input or output defined in C itself
- Requires library stdio
- Two methods:
  - Character based (no format specifiers) – character by character I/O
    - getchar() - input
    - putchar(c) - output
  - Formatted - standard I/O
    - scanf(stuff goes in here) - input - white space is important!!!
    - printf(stuff goes in here) - output
    - Format Specifiers (% before specifier) – see next slide

```
#include <stdio.h>
int main()
{     /* check1.c */
      int x;
      scanf("%d\n", &x);
      printf("x=%d\n", x);
}
```

Format Strings

Conversion Character	Displays Argument (Variable's Contents) As
%c	Single character
%d, %i	Signed decimal integer (int)
%e	Signed floating-point value in E notation
%f	Signed floating-point value (float)
%g	Signed value in %e or %f format, whichever is shorter
%o	Unsigned octal (base 8) integer (int)
%s	String of text
%u	Unsigned decimal integer (int)
%x	Unsigned hexadecimal (base 16) integer (int)
%%	(percent character)

When programming in C, you use format strings — the percent sign and a letter, for the most part — as placeholders for variables you want to display. The above table shows the format strings and what they display...

Codes for Formatted I/O: "%-+ 0*w.pmc*"

`-`	left justify
`+`	print with sign
space	print space if no sign
`0`	pad with leading zeros
w	min field width
p	precision
m	conversion character:

`h` short, `l` long, `L` long double

c	conversion character:

`d,i`	integer	`u`	unsigned
`c`	single char	`s`	char string
`f`	double (`printf`)	`e,E`	exponential
`f`	float (`scanf`)	`lf`	double (`scanf`)
`o`	octal	`x,X`	hexadecimal
`p`	pointer	`n`	number of chars written

`g,G` same as `f` or `e,E` depending on exponent

printf examples

```
int a = 42, b = 11;
printf("%d\t%d\n", a, b);
```

Output: “a b”

The values of the two integers a and b are printed with a tab (\t) between them

```
printf("%3d %6d\n", a, b);
```

Output “ 42 11”

[SPACE]42[Explicit SPACE from printf][SPACE*4]11

NOTE: Each % construction in the first argument of printf is paired with the corresponding second argument, third argument, etc.; they must match up properly by number and type, or you will get wrong answers or an error.

```
printf("\na=%f\nb=%f\nc=%f\nPI=%f",a,b);   //ERROR
```

printf examples

```
int y = 25;
printf("%d\n", y);
printf("%i\n", y);
printf("%4d\n", y);
printf("%1d\n", y);
printf("%05d\n", y);
```

output:
25
25
25
25
00025

```
float x = 3.14159;
printf("%4.3f\n", x);
printf("%4f\n", x);
printf("%.3f\n", x);
```

output:
3.142
3.141590
3.142

printf vs scanf

- Both format I/O
- Both manipulate “standard I/O” location
- printf - output
  - Converts values to character form according to the format string
- scanf - input
  - Converts from characters according to the format string, and followed by pointer arguments indicating where the resulting values are stored

scanf Examples

```
int day, month, year;
scanf("%d/%d/%d", &month, &day,
&year);
Input: 01/29/64

int i;
scanf("%i%%", &i);
Input: 23%

int i; long l;
scanf("%d  %ld", &i, &l);
Input: -23 200

double d;
scanf("%lf", &d);
Input: 3.14
```

```
char s[128];
scanf("%9s", &s);
Input: VeryLongString
Result: s=“VeryLongS”

int i;
scanf("%*s %i", &i);
Input: Age: 29
Result: i=29

int i1, i2;
scanf("%2i", &i1);
scanf("%2i", &i2);
Input: 2345
Result: i1=23, i2=45
```

scanf Cautions

- scanf and white space can be very tricky!
- When troubleshooting, make sure you read about and understand how scanf handles whitespace
- http://www.cplusplus.com/reference/cstdio/scanf

White Space

- White space is any “blank” space
  - Space (‘ ‘)
  - Tabs (‘\t’)
  - Newlines (‘\n’)
  - Carriage return (‘\r’)
  - Vertical tab (‘\v’)
  - Form feed (‘\f’)

scanf whitespace Examples

```
// Example 1
int x, y;
scanf("%d %d", &x, &y);

Input:
1
2

Result:
x: 1
y: 2
Stream: empty
```

```
// Example 2
int x, y;
scanf("%d %d", &x, &y);

Input:
1, 2

Result:
x: 1
y: unchanged
Stream: ", 2"

// successfully parsed 1 into x
but stopped at ,
```

Scanf examples

Letter	Type of Matching Argument	Auto-skip; Leading White-Space	Example	Sample Matching Input
%	% (a literal, matched but not converted or assigned)	no	int anInt; scanf("%i%%", &anInt);	23%
d	int	yes	int anInt; long l; scanf("%d %ld", &anInt, &l);	-23 200
i	int	yes	int anInt; scanf("%i", &anInt);	0x23
o	unsigned int	yes	unsigned int aUInt; scanf("%o", &aUInt);	023
u	unsigned int	yes	unsigned int aUInt; scanf("%u", &aUInt);	23
x	unsigned int	yes	unsigned int aUInt; scanf("%x", &aUInt);	1A
a, e, f, g	float or double	yes	float f; double d; scanf("%f %lf", &f, &d);	1.2 3.4
c	char	no	char ch; scanf(" %c", &ch);	Q
s	array of char	yes	char s[30]; scanf("%29s", s);	hello
n	int	no	int x, cnt; scanf("X: %d%n", &x, &cnt);	X: 123 (cnt==6)

[	array of char	no	char s1[64], s2[64];
			scanf(" %[^\n]", s1);
			scanf("%[^\t] %[^\t]", s1, s2);
				
				Hello World
				field1 field2

CSE 2421

File Operations

File Operation Summary

- Open/Close files
  - fopen() – open a stream for a file
  - fclose() – closes a stream
- One character at a time:
  - fgetc() – similar to getchar()
  - fputc() – similar to putchar()
- One line at a time:
  - fprintf()/fputs() – similar to printf()
  - fscanf()/fgets() – similar to scanf()
- File errors
  - perror() – reports an error in a system call

- Stay tuned for more in-depth coverage

stdio.h definitions

1) FILE – a structure that holds buffered file stream info
2) fpos_t – a variable type suitable to store a position in a file
3) NULL – value of a null pointer constant
4) EOF – negative integer which indicates end-of-file has been reached
5) FOPEN_MAX – integer which represents the maximum number of files that the system can guarantee that can be opened simultaneously
6) FILENAME_MAX – integer which represents the longest length of a char array
7) stderr/stdin/stdout – pointers to FILE types which correspond to the standard streams

standard file handles

- When a program begins, there are already three available streams which are predefined and need not be opened explicitly and are of type “pointer to FILE”
  - standard input (stdin)
  - standard output (stdout)
  - standard error (stderr)
- Files are associated with streams and must be opened to be used.
- The point of I/O within a file is determined by the file position.
- When a file is opened, the file position points to the beginning of the file (unless the file is opened for an append operation in which case the position points to the end of the file).
- The file position follows read and write operations to indicate where the next operation will occur.
- When a file is closed, no more actions can be taken on it until it is opened again.
- Exiting from the main function causes all open files to be closed.

FILE open/close

1) `FILE *fopen(const char *filename, const char *mode);`
    1) Mode… (lots more!)
    2) r – read text mode
    3) w – write text mode (truncates file to zero length or creates a new file)
    4) If the file does not exist and it is opened with read mode (r), then the open fails ; need to check for this
2) `int fclose(FILE *stream);`
    1) Closes the stream.
    2) If successful, it returns zero.
    3) On error it returns EOF.
3) `void perror(const char *str);`
    1) Prints a descriptive error message to stderr. First the string str is printed followed by a colon then a space (your error message). Then an error message based on the current setting of the variable errno is printed (system error message).

Open/Read a File – one char at a time

```
// complete program; reading a file one character at a time
#include <stdio.h>
#include <stdlib.h>

int main(){
    char ch;

    FILE *fp = fopen("file","r");
    if (fp==NULL) {
        perror("Error while opening the file.\n");
        exit(EXIT_FAILURE);
    }

    printf("File contents:\n\n");
    while ((ch=fgetc(fp))!=EOF) {
        printf("%c",ch);
    }

    fclose(fp);
    return 0;
}
```

fgets vs fscanf

1) `char *fgets(char *str, int n, FILE *stream);`
   1) Reads a line from the specified stream and stores it into the string pointed to by str.
   2) It stops when either (n-1) characters are read, the newline character is read, or the end-of-file is reached, whichever comes first.
   3) The newline character is copied to the string.
   4) A null character is appended to the end of the string.
   5) On error a null pointer is returned. If the end-of-file occurs before any characters have been read, the string remains unchanged.

2) `int fscanf(FILE *stream, const char *format, ...);`
   1) Reading an input field (designated with a conversion specifier) ends when an incompatible character is met, or the width field is satisfied.
   2) On success the number of input fields converted and stored are returned. If an input failure occurred, then EOF is returned.
   3) Returns EOF in case of errors or if it reaches EOF

fprintf and feof

1) `int fprintf(FILE *stream, const char *format, ...);`
   1) sends formatted output to a stream
   2) Just like printf, but puts file pointer as first argument
   3) In lab2p2:
   4) fprintf(outfp, “Your enciphered message is %s\n”,input);

2) `int feof(FILE *stream);`
   1) Tests the end-of-file indicator for the given stream
   2) If the stream is at the end-of-file, then it returns a nonzero value. If it is not at the end of the file, then it returns zero.

fgetc/fputc

1) `int fgetc(FILE *stream);`
   1) Gets the next character (an unsigned char) from the specified stream and advances the position indicator for the stream.
   2) On success the character is returned.
   3) If the end-of-file is encountered, then EOF is returned and the end-of-file indicator is set.
   4) If an error occurs then the error indicator for the stream is set and EOF is returned.

2) `int fputc(int char, FILE *stream);`
   1) Writes a character (an unsigned char) specified by the argument char to the specified stream and advances the position indicator for the stream.
   2) On success the character is returned.
   3) If an error occurs, the error indicator for the stream is set and EOF is returned.

open/read/write/close example

```
// example - abridged program - copy a file, character by character
char ch, chout;
FILE *fpin, *fpout;
fpin = fopen("input_file","r");
if (fpin==NULL) {
    perror("Error while opening the input file.\n");
    exit(EXIT_FAILURE);
}
fpout = fopen("output_file","w");
if (fpout==NULL) {
    perror("Error while opening the output file.\n");
    exit(EXIT_FAILURE);
}
while( ( ch = fgetc(fpin) ) != EOF ) {
     chout = fputc(ch,fpout);   // ret char if success, otherwise EOF
     if (chout == EOF) break;
}
fclose(fpin);
fclose(fpout);
```

Example

```
// abridged program; reading a file one
chunk at a time

int chunk_length=100;
char string[chunk_length];

do {
  fgets(string,chunk_length,fp);
  puts(string);
} while (!feof(fp));
```

CSE 2421

Loops/Conditionals

Loop Constructs

```
for (init; cond; modify) {
      statement(s); }
while (cond) {
      statement(s); }
do {
      statement(s);
} while (cond);
```

- Statement(s) only execute when the condition is TRUE.
- Notice the semi-colon locations.
- Do/while always executes at least once (not necessarily true for the other two constructs).
- For loop → initialize a value, test the condition, if condition is true, execute the statements (if condition is false, exit the loop), modify the value, test the condition, if the condition is true, execute the statements, etc.

Loop Example 1

```
#include <stdio.h>

int main ()  {
    int n, sum;
    sum = 0;

    // n++ also n+=1 also ++n
    for (n = 1; n <= 10; n=n+1)
    {
            sum = sum + n;
    }
    // printf is outside the for loop
    printf("\n The sum of integers from 1 to 10 is %d, have a nice day", sum);
    return 0;
}
```

Output: The sum of integers from 1 to 10 is 55, have a nice day

Loop Example 2

```
int sum=0, n=1;
while (n<=10) {
        sum=sum+n;
        n=n+1;
}
==========================
int sum=0, n=0;
do {
        n=n+1;
        sum=sum+n;
} while (n<10);
```

Loop Example 2

```
int sum=0, n=1;
while (n<=10) {
        sum=sum+n;
        n=n+1;
}
```

n=
1
2
3
4
5
6
7
8
9
10
Still 55

```
==========================
int sum=0, n=0;
do {
        n=n+1;
        sum=sum+n;
} while (n<10);
```

Loop Example 2

```
int sum=0, n=1;
while (n<=10) {
        sum=sum+n;
        n=n+1;
}
==========================
int sum=0, n=0;
do {
        n=n+1;
        sum=sum+n;
} while (n<10);
```

n=
0 (extra iteration)
1
2
3
4
5
6
7
8
9
10
Still 55

Loop Example 2.1

```
int sum=0, n=11;
while (n<=10) {
        sum=sum+n;
        n=n+1;
}
==========================
int sum=0, n=11;
do {
        n=n+1;
        sum=sum+n;
} while (n<10);
```

Loop Example 2.1

```
int sum=0, n=11;
while (n<=10) {
        sum=sum+n;
        n=n+1;
}
```

n=11 (while condition fails)

Sum=0

```
==========================
int sum=0, n=11;
do {
        n=n+1;
        sum=sum+n;
} while (n<10);
```

n=11

evaluate inside do once:
Sum=11

Loop Example 3

```
#include <stdio.h>
int main() {
    int x;
     x = 0;
     do {
          printf( "Hello, world!\n" );
     }
     while ( x != 0 );
     getchar();
     return 0;
}
/*
     "Hello, world!" is printed one time even though the condition
is false
*/
```

Loop Example 4

```
#include <stdio.h>
int main() {
    int x = 0;
    while ( x < 10) {
        printf("%d\n", x );
        x++;
    }
    getchar();
    return 0;
}
```

Output:
0
1
2
3
4
5
6
7
8
9

Loop Example 5

```
#include <stdio.h>
int main()
{
    int x;
    for (x = 0; x < 10; x++) {
        printf("%d\n", x );
    }
    getchar();
    return 0;
}
```

Output:
0
1
2
3
4
5
6
7
8
9

1 line Loops

- If a loop statement is only going to contain one line you can leave off the { }

```
while(x < 10)
printf("%d", x++);

for(int i = 0; i < 10; ++i)
printf("%d", i);
```

*note: best practice is to use { } for clarity, but this shorthand is still valid

Break, Continue

- Keywords that are very important to looping are *break* and *continue*.
- *break* exits the innermost loop regardless of loop conditions
- *continue* causes the loop to stop its current iteration, update itself (in the case of FOR loops) and begin to execute again from the top.

Jump Statement

- Goto plus a labeled statement
  - goto identifier ;
  - identifier: statement;
- Don’t have to pre define or declare the identifier
- A statement label is meaningful only to a goto statement
  - Otherwise its ignored
- A jump-statement must be contained in one function
- Identifier labels have their own name space so the names do not interfere with other identifiers.
- Labels cannot be redeclared/reused in a function
- Best practice to use break, continue, and return statement in preference to goto whenever possible.
  - Special cases for goto. I.e. exiting multiple levels of loops

Break Example

```
/* playing checkers */

while (true) {
    if(has_won() || wants_to_quit() == TRUE){
    break;
  }

    take_turn(player1);

    if (has_won() || wants_to_quit() == TRUE){
    break;
  }
    take_turn(player2);
}
```

Continue Example

```
/* playing monopoly */
for (player = 1; has_won() == FALSE; player++) {
    if (player > total_number_of_players) {
            player = 1;
  }

    if (is_bankrupt(player)) {
            continue;
  }
    take_turn(player); }
```

Goto Example

```
#include <stdio.h>
int main() {
   int i, j;
   for ( i = 0; i < 10; i++ ) {
      printf("Outer loop. i =%d\n",i);
      for ( j = 0; j < 3; j++ ) {
         printf("Inner loop. j = %d\n",j);
            if ( i == 1 )
               goto stop;
      }
   }
   printf("Loop exited. i = %d\n", i );
stop:
   printf("Jumped to stop. i = %d\n", i );
   return 0;
}
Output:
Outer loop. i=0
Inner loop. j=0
Inner loop. j=1
Inner loop, j=2
Outer loop. i=1
Inner loop, j=0
Jumped to stop. i=1
```

goto Fail

iOS 7.0.6 - security flaw in SecureTransport (iOS SSL)

```
....
hashOut.data = hashes + SSL_MD5_DIGEST_LEN;
hashOut.length = SSL_SHA1_DIGEST_LEN;
if ((err = SSLFreeBuffer(&hashCtx)) != 0)
    goto fail;
if ((err = ReadyHash(&SSLHashSHA1, &hashCtx)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
    goto fail;
    goto fail;  /* MISTAKE! THIS LINE SHOULD NOT BE HERE */
if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
    goto fail;

err = sslRawVerify(...);
...
```

The result is that the code leaps over the vital call to `sslRawVerify()`, and exits the function and reports success.

https://nakedsecurity.sophos.com/2014/02/24/anatomy-of-a-goto-fail-apples-ssl-bug-explained-plus-an-unofficial-patch/

CSE 2421

Conditionals

If, else-if, switch-case conditional statements

```
if ( TRUE ) {
    /* Execute these  stmts if
     TRUE */ }
else {
    /* Execute these stmts if
    FALSE */ }
```

```
if (condition) {
     statement(s);  }
else if (condition) {
     statement(s);  }
else {
     statement(s);  }
```

```
switch ( <variable> ) {
    case this-value:          /* Note the :, not a ; */
        Code to execute if <variable> == this-value;
        break;
    case that-value:
        Code to execute if <variable> == that-value;
        break;
...  default:
        Code to execute if <variable> does not equal
         the value following any of the cases… break; }
```

- SWITCH NOTES:
- Notice, no {} blocks within each case.
- Notice the colon for each case and value.
- The “condition” of a switch statement is a const value.
- The default case is optional, but it is wise to include it as it handles any unexpected cases.
- Fall throughs are allowed
- Chooses first match…

Else If Example

```
#include <stdio.h>
int main() {
    int age;
    printf( "Please enter your age" );  scanf( "%d", &age );
    if ( age < 100 ) {
        printf ("You are young!\n" );     }
    else if ( age == 100 ) {
        printf("You are old\n" );
    }
    else {
        printf("You are really old\n" );  }
    return 0;
}
```

Switch Case

```
switch ( x ) {
case 'a':
      /* Do stuff when x is 'a' */
      break;
case 'b':
case 'c':
case 'd':
      /* Fallthrough technique...
           cases b,c,d all use this code */
      break;
default:
      /* Handle cases when x is not
           a,b,c  or d. ALWAYS have a
           default  case*/
      break; }
```

Switch Case Example

```
#include <stdio.h>
void playgame() { printf( "Play game called" ); }
void loadgame() { printf( "Load game called" ); }
void playmultiplayer() { printf( "Play multiplayer game called" ); }
int main() {
    int input;
    printf( "1. Play game\n" );
    printf( "2. Load game\n" );
    printf( "3. Play multiplayer\n" );
    printf( "4. Exit\n" );
    printf( "Selection: " );
    scanf( "%d", &input );
    switch ( input ) {
        case 1:
            playgame();
            break;
        case 2:
            loadgame();
            break;
        case 3:
            playmultiplayer();
            break;
        case 4:
            printf( "Thanks for playing!\n" );
            break;
        default:
            printf( "Bad input, quitting!\n" );
            break; }
    getchar();
    return 0; }
```

This type of menu can be useful when initially creating complex programs

Boolean issues

- Every boolean test is an implicit comparison against zero (0).
- However, zero is not a simple concept. It represents:
  - the integer zero for all integral types
  - the floating point 0.0 (positive or negative)
  - the nul character (‘\0’)
  - the null pointer
- In order to make your intentions clear, explicitly show the comparison with zero for all scalars, floating-point numbers, and characters.

Boolean issues

- int i; if (i) is better seen as if (i == 0)
- float x; if (!x) is better seen as if (x != 0.0)
- char c; if (c) is better seen as (c == '\0')

- An exception is made for pointers, since 0 is the only language-level representation for the null pointer.

- The symbol NULL is not part of the core language - you have to include a special header file to get it defined

Boolean issues

- Write an INFINITE LOOP
  - for (;;) ...
  - while (1) ...
- Reminder
  - 0 = False
  - Non 0 = True

The Comma Operator

- Used to link related expressions together
- Evaluated from left to right
- The value of the right most expression is the value of the combined expression
- Example:
  - value = (x = 10, y = 5, x + y);
  - i = a, b, c; // stores a into i
  - i = (a, b, c); // stores c into i
- Parentheses are necessary!
- For loop:
  - for (n=1, m=10; n<=m; n++, m--)
- While:
  - while (c=getchar(), c!= ‘10’)
- Exchanging values:
  - t=x, x=y, y=t;

CSE 2421

Enums

Enumerated Data Types

- An enumerated type is one whose values are symbolic constant rather than literal.
- Declaration example:
  - **enum** Jar_Type {CUP, PINT, QUART, HALF_GALLON, GALLON};
- The above example declares a type called `Jar_Type`
- Variables of this type are declared like this:
  - **enum** Jar_Type milk_jug;

Enumerated Data Type (cont)

- Variables declared with an enumerated type are actually stored as integers.
- Internally, the symbolic names are treated as integer constants
  - `enum Jar_Type {CUP, PINT, QUART, HALF_GALLON, GALLON};`
  - By default, CUP=0, PINT=1, QUART=2, etc.
- Caution: don’t mix them indiscriminately with integers – even though it is viable.
  - milk_jug = -623;
  - int a = PINT;

Enumerated Data Types

- If there is only one declaration of variables of a particular enumerated type (i.e. no type name), both statements may be combined:
  - `enum { CUP, PINT, QUART, HALF_GALLON, GALLON} milk_jug, gas_can, medicine_bottle;`
- milk_jug, gas_can, and medicine_bottle are now all instances of the enum.
- No more instances can be made because no type name was given to it
- Not the best usage of this feature

Enumerated Types Example

```
#include <stdio.h>
enum location{east=1, west=2, south=3, north=4};
main() {
    enum location direction;
    direction = east;
    if( direction == east ) {
        printf("Cannot go east\n");
     }
    enum month { jan = 1, feb=2, mar=3, apr=4,
         may=5, jun=6, jul=7,aug=8, sep=9,
         oct=10, nov=11, dec=12 } this_month;

    this_month = feb;
    printf("month : %d\n",this_month);
}
```

Output:

Cannot go east\n
month: 2\n